

Weeks 3/4- Scanning and Enumeration

6COSC019W

Dr Ayman El Hajjar
University of Westminster

Academic Year: 2025/26

1 Lab environment

VMs IP Reference

- ➡ Example IP addresses in this lab:
 - ↳ OWASP VM: 192.168.144.129
 - ↳ Kali VM: 192.168.144.128
- ➡ Your IP addresses may be different due to DHCP.
- ➡ Always use your own VM IPs.

STOP and READ

- ➡ Make sure your lab environment is set up as described in **Lab 1**.
- ➡ This includes: importing the VMs, configuring the network, and running the studentid script.
- ➡ If you are unsure, refer back to the Lab Environment Setup guide.

```
➤- ./studentid.sh
```

- ➡ Download the module files while Kali is connected to **NAT**.

```
➤- cd Desktop
```

```
➤- git clone https://github.com/azelhajjar/6COSC019W.git
```

- ➡ Some activities require the OWASP VM.
- ➡ Use **NAT** for Internet-based targets such as cwsenario.uk.
- ➡ Use **Host-Only** for local targets such as the OWASP VM.
 - ↳ If the IP does not update after changing the adapter:

```
➤- sudo ifconfig eth0 down
```

```
>- sudo ifconfig eth0 up
```

```
>- sudo ifconfig eth0
```

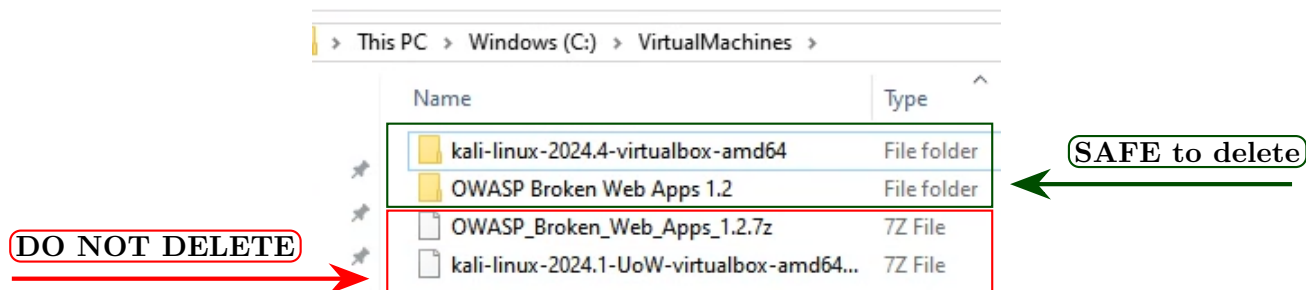


Figure 1.1: VirtualMachines folder contents

2 Finding Files and Folders

- ➡ DirBuster is a tool used to discover files and directories on a web server using a wordlist.
- ➡ It attempts common file and folder names and checks the server responses.

DirBuster Overview

- ➡ DirBuster works as both a crawler and a brute-force tool.
- ➡ It follows links and also tests paths from a dictionary.
- ➡ It uses HTTP response codes to identify resources.
- >- 200 OK File exists
- >- 404 Not Found File does not exist
- >- 301 Redirect
- >- 401 Authentication required
- >- 403 Access forbidden

- ➡ We will first scan the OWASP VM while Kali is connected to **Host-Only**.

↳ Target IP: **192.168.144.129**

- ➡ We will use the **modulewordlist.lst** file from the module repository.

- ➡ Launch DirBuster from the Kali menu (see Figure 2.1).

↳ [1] Set the target URL to: **http://192.168.144.129**

↳ [2] Set the number of threads to **20**.

↳ [3] Select **List-based brute force**.

↳ [4] Browse to the **modulewordlist.lst** file and load it.

↳ [5] Uncheck **Be Recursive**.

↳ [6] Click **Start**.

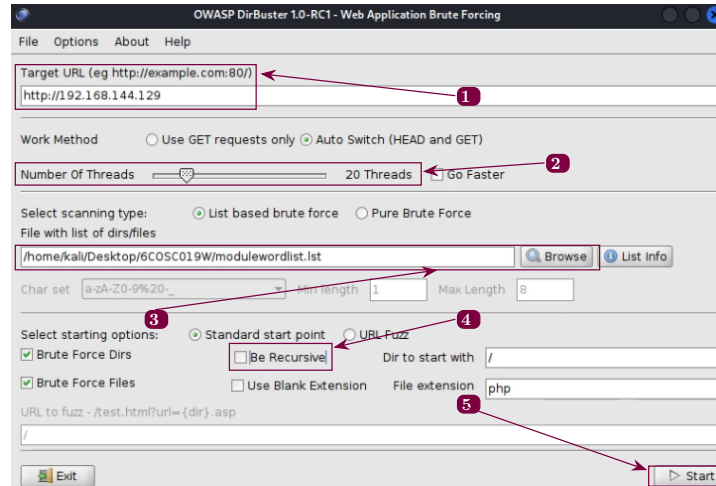


Figure 2.1: DirBuster configuration for scanning the OWASP VM

➡ Review the results tab and note discovered files and folders.



Now switch Kali to **NAT** and scan the live website.

↳ Target URL: **https://cwscenario.uk**

↳ Set the number of threads to **5**.

↳ Do not increase the thread count on live systems.

↳ Use the same dictionary and settings as before.

➡ Review the results and explore any discovered paths in the browser.

Optional Command-Line Alternative

➡ You can also use the **dirb** command-line tool.

```
> man dirb
> dirb https://cwscenario.uk/ modulewordlist.lst
> dirb http://192.168.144.129 modulewordlist.lst
> dirb http://192.168.144.129 modulewordlist.lst -v
> dirb http://192.168.144.129 modulewordlist.lst -o results.txt
```

➡ Record any useful findings for later activities.

3 Scanning activity

➡ One of the most important activities one can do in the reconnaissance phase is to gather information using Scanning and Enumeration methods.

PLEASE READ BEFORE YOU START THIS SECTION

➡ We will be mainly using nmap however there are many other tools we can be used. For other tools pre-installed on Kali about OSINT, Information Gathering, and tools to enumerate networks ports and addresses, services and protocols, click on the Kali home button and select → **01 - Information Gathering**.

➡ Nmap is probably the most used port scanner in the world. It can be used to identify live hosts, scan TCP and UDP open ports, detect firewalls, get versions of services running in remote hosts, and even, with the use of scripts, find and exploit vulnerabilities.

➡ We first need to see if the server is answering to a ping or if the host is up.

```
❏ nmap 192.168.144.129
```

➡ **This is not what we wanted!** At least at this stage, this command completed a port scan using a TCP SYN scan, while we only wanted to ping the device to see if it is live.

```
❏ man nmap
```

➡ According to the nmap help page (Fig.3.1), the default nmap scan will conduct a TCP SYN scan. To disable port scanning while pinging the IP address, we need to use the **-sn** option. This disables the port scan and only checks whether the host is live.

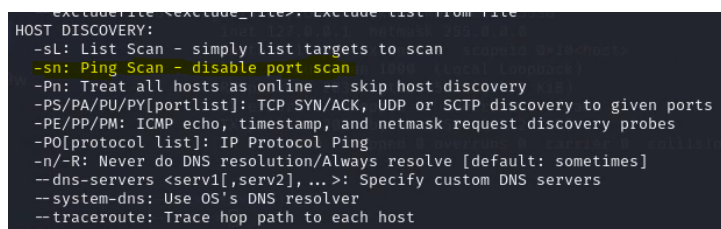
A screenshot of the nmap manual page. The text is as follows:
ExcludeFile <exclude_file>: Exclude list from file
HOST DISCOVERY:
-sL: List Scan - simply list targets to scan
-sn: Ping Scan - disable port scan
-Pn: Treat all hosts as online -- skip host discovery
-PS/PA/PY[portlist]: TCP SYN/ACK, UDP or SCTP discovery to given ports
-PE/PP/PM: ICMP echo, timestamp, and netmask request discovery probes
-PO[protocol list]: IP Protocol Ping
-n/-R: Never do DNS resolution/Always resolve [default: sometimes]
--dns-servers <serv1[,serv2],...>: Specify custom DNS servers
--system-dns: Use OS's DNS resolver
--traceroute: Trace hop path to each host

Figure 3.1: Nmap manual - Disabling port scan

➡ To check if the IP address is responding and the server is up:

```
❏ nmap -sn 192.168.144.129
```

➡ The OWASP server responded that it is up. If you don't get this result, make sure the OWASP VM is running and connected to the Host-Only network.

➡ In this command, with the **-sn** parameter, we instructed **nmap** to conduct a ping scan to check if the server was responding using ICMP.

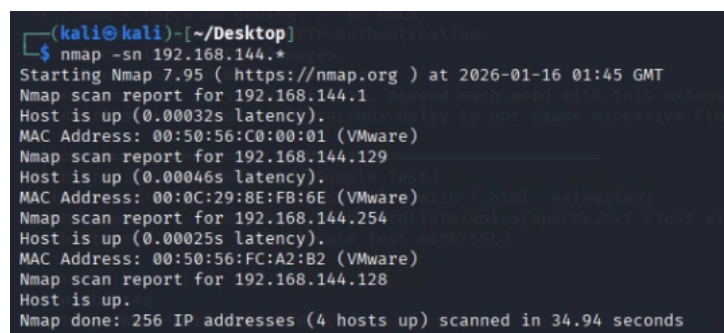
3.1 Identifying live hosts Nmap

➡ In the previous section, we used nmap with the **-sn** option and identified that the **192.168.144.129** IP address is live and responding.

➡ Let's begin the process of locating live hosts (active machines). We already know the OWASP machine IP address.

```
❏ nmap -sn 192.168.144.*
```

➡ Using the asterisk ***** instead of a specific value tells Nmap to scan all hosts in this IP range (Fig.3.2).



```
(kali㉿kali)-[~/Desktop]
$ nmap -sn 192.168.144.*
Starting Nmap 7.95 ( https://nmap.org ) at 2026-01-16 01:45 GMT
Nmap scan report for 192.168.144.1
Host is up (0.00032s latency).
MAC Address: 00:50:56:C0:00:01 (VMware)
Nmap scan report for 192.168.144.129
Host is up (0.00046s latency).
MAC Address: 00:0C:29:8E:FB:6E (VMware)
Nmap scan report for 192.168.144.254
Host is up (0.00025s latency).
MAC Address: 00:50:56:FC:A2:B2 (VMware)
Nmap scan report for 192.168.144.128
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 34.94 seconds
```

Figure 3.2: Network scan results for active hosts. Host machine IP: **192.168.144.1**, Kali VM: **192.168.144.128**, OWASP VM: **192.168.144.129**, VMware internal network service: **192.168.144.254**.

➡ We can also use 0/24 instead of the *****. For example, **nmap -sn 192.168.144.0/24** will instruct Nmap to scan all hosts on the 192.168.144.0 subnet.

➡ In a real live network environment, you will typically find many more IP addresses than in this lab setup.

3.2 TCP Scans

➡ With the knowledge of the victim's network range and the active machines, we'll proceed with the port scanning process to retrieve the open TCP ports and access points.

➡ Since these type of scans are using the TCP transport layer protocol, it leverages the three way handshakes that TCP uses. Depending on what we are looking for and how quiet or loud we need to be, we can instruct nmap to use one the various TCP scans methods below, from the quietest, to the loudest.

➡ **TCP NULL scan-** This is the quietest TCP scan. In this scan, nmap sends a packet with no flags set (Basically a TCP flag that is not set to any of the TCP possible flags SYN, ACK, RST, or FIN). The goal is to see how the target system responds. If the port is open, there should be no response; if it's closed, an RST response will be sent.

```
❏ nmap -sn 192.168.144.129
```

➡ This is the same as the one we used before. The ICMP protocol in the network

layer needs a transport layer protocol to deliver its ICMP request. The **n** stands for **null**.

➡ **TCP FIN Scan-** This is a quiet TCP scan. In this scan, Nmap sends a FIN flag to close a connection that doesn't actually exist. In this case, the server would respond with a filtered open response meaning it is open but doesn't know how to respond to the request.

```
➤ nmap -sF 192.168.144.129
```

↳ The responses will indicate whether services are available and open on different ports. If a response is received, it means the port is open. The **F** stands for the **FIN** flag.

➡ **TCP SYN Scan-** This TCP scan is moderately quiet. It is commonly referred to as “**half open**” because a full TCP connection is not established. This type of scan was originally developed to be stealthy and evade IDS systems, although most modern systems have adapted to detect it. Open ports reply with a SYN/ACK, whereas closed ports respond with an RST/ACK.

```
➤ nmap -sS 192.168.144.129
```

↳ This is also the default Nmap port scan. It is often sufficient to identify running services. The **S** stands for **SYN** scan.

➡ **TCP ACK Scan-** This is a moderately loud TCP scan. In this scan, nmap attempts to determine the firewall access control list (ACL) rules or identify whether **stateless inspection** is being used. **Stateless inspection** is a method firewalls use to inspect packets without looking at their contents.

```
➤ nmap -sA 192.168.144.129
```

↳ This scan does not directly check for open ports. Instead, it checks whether ports are filtered based on firewall behaviour. You will notice that the results do not show any specific open ports. However, the terms **unfiltered** or **ignored state** indicate that the firewall is not blocking port requests. The **A** stands for **ACK**.

➡ **TCP Connect Scan-** This is the loudest TCP scan, but also the most reliable. It can be easily logged and detected because a full connection is established. Open ports respond with a SYN/ACK, while closed ports reply with an RST/ACK.

```
➤ nmap -sT 192.168.144.129
```

↳ As it completes the handshake and is very loud, it is usually used only if other scans did not provide enough information due to network restrictions, such as a firewall. If other scans have provided sufficient information, this option is better avoided. The **T** stands for **connect**.

3.3 UDP scan

➡ Up until this point, all of our scans have been for TCP ports. Some services and ports use UDP to communicate to the outside world such as NTP (port 123) and SNMP (port 161). To find these ports and services, we need to do a UDP scan. We can do this with the **-sU** option.

```
➤- nmap -sU 192.168.144.129
```

➡ **A UDP scan will take very long time and longer than TCP scans.** This is because UDP is a connectionless protocol and the lack of the three way handshake makes it resource intensive, as it goes through each port and does not require acknowledgments, unlike TCP scans which skip ports that do not respond.



If you have limited time, such as when in class, you can leave the command below and run it on your own environment when you have more time.

↳ It took **28 minutes** on my machine.

4 Enumeration

➡ Enumeration is a phase in the information-gathering process during cybersecurity assessments.

➡ It involves extracting detailed information about a target system or network to understand its configuration, services, and potential vulnerabilities.

➡ The goal of enumeration is to create a comprehensive profile of the target, aiding in the identification of security weaknesses.

➡ This includes identifying versions of servers, services, and operating systems the target uses.

4.1 Enumeration for Operating systems

➡ At this point of the information gathering process, we should now have documented a list of IP addresses, active machines, and open ports identified from the target organisation. The next step in the process is determining the running operating system of the active machines in order to know the type of systems we're pen-testing.

➡ The next step is to try to identify the **Operating System (OS)** the server is using. We issue the following command with the **-O** option to enable the OS detection feature.

```
➤- nmap -O 192.168.144.129
```

➡ The **-O** option will compare the fingerprint response of the server with Nmap's database of OS fingerprints. If it finds a similar fingerprint, it highlights it as a match.

➡ In this case, Nmap returned a family of OS matches: Kernel 2.6.X with possible versions between 2.6.17 and 2.6.36 (Fig.4.1).

➡ It can also return an exact OS match (e.g. Linux Kernel 2.6.14), a partial match (e.g. 2.x or 3.x Linux kernel), or no match at all.

```
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6
OS details: Linux 2.6.17 - 2.6.36
Network Distance: 1 hop
```

Figure 4.1: Operating System enumeration results on the OWASP VM.

! This is a very noisy command and an Intrusion Detection System (IDS) or firewall will likely detect it.

4.2 Enumeration for services

➡ Determining the services running on specific ports will ensure a successful pentest on the target network. It will also remove any doubts left resulting from the OS fingerprinting process.

➡ We will need to conduct services enumeration by attempting to detect services running on the OWASP machine using the **-sV** option.

```
➤ nmap -sV 192.168.144.129
```

➡ The Versions detection (also called Versions fingerprinting) option **-sV** will return the services running on the system and their version as shown in Figure 4.2.

```
(kali@kali)~$ nmap -sV 192.168.144.129
Starting Nmap 7.92 ( https://nmap.org ) at 2026-01-16 02:05 GMT
Nmap scan report for 192.168.144.129
Host is up (0.0016s latency).
Not shown: 991 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh      OpenSSH 5.3p1 Debian 3ubuntu4 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.2.14 ((Ubuntu) mod_mono/2.4.3 PHP/5.3.2-1ubuntu4.30 with Suhosin-Patch proxy_html/3.0.1 mod_python/3.3.1 Python/2.6.5 mod_ssl/2.2.14 OpenSSL/1.0.1)
139/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
143/tcp   open  imap     Courier Imapd (released 2006)
443/tcp   open  ssl/http Apache httpd 2.2.14 ((Ubuntu) mod_mono/2.4.3 PHP/5.3.2-1ubuntu4.30 with Suhosin-Patch proxy_html/3.0.1 mod_python/3.3.1 Python/2.6.5 mod_ssl/2.2.14 OpenSSL/1.0.1)
445/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
5801/tcp  open  java-object Java Object Serialization
8080/tcp  open  http     Apache Tomcat/Coyote JSP engine 1.1
8081/tcp  open  http     Jetty 6.1.25
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
_
SF-Port5001-TCP-V=7.95XI=7MD=1/16XTime=69699CF5XP=x86_64-pc-linux-gnuXr(NU
SF-LL,4,"xac\xed\x05");
MAC Address: 00:0C:29:A8:F3:6E (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 84.09 seconds
```

Figure 4.2: Version Detection for OWASP. As you can see, we now know in addition to the port details, the version of the service running behind the port.

What can we do with enumeration information?

➡ When conducting enumeration for services, we are able to find specific versions for services and operating systems running on our target. We can then look for known vulnerabilities that exist for those services. We can use any **Common Vulnerabilities and Exposures** database to search for such vulnerabilities.

➡ Even a Google search will reveal vulnerabilities for services. Try it now: while connected to the Internet, open a browser and search for "OpenSSH 5.3p1 CVE

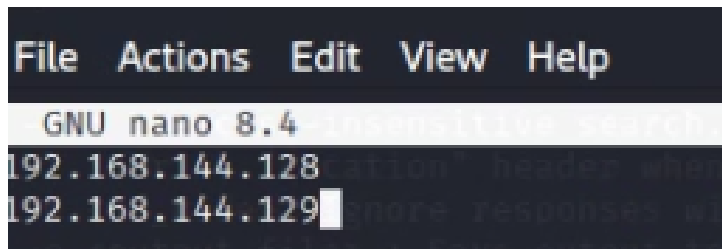


Figure 5.1: scanlist file with two IP addresses

vulnerabilities" and see what comes up.

5 Options

➡ Before we continue with nmap and look at more advanced methods to use it, we need to look at some of the options that gives nmap its strength and makes it the most popular information gathering tool.

5.1 List of targets

➡ Many times we want to scan a list of IP addresses and not an entire subnet. In a real life scenario, you would find many IP addresses in an environment.

➡ We can use any text editor and create a list of IP addresses and "feed" it to nmap. This is the list of IP addresses that you want to scan. You can either use mousepad, the GUI-based text editor in Kali, or you can use nano, the terminal-based text editor.

```
➤- mousepad scanlist.txt
```

↳ or

```
➤- nano scanlist.txt
```

➡ Both commands will create the file once you save it after editing.

➡ **Add in the file as many IP addresses as you need.**

➡ For example, you can put the OWASP and the Kali IPs, each on its own line (Fig.5.1).

➡ Once you save the file, assuming you are in the same folder (use the absolute path if you are not in the same folder), you can scan using the **-iL** option to provide the name of the IP addresses file.

```
➤- nmap -iL scanlist.txt
```

➡ This will scan both IP addresses in the scanlist file.

5.2 Spoof or Decoy Scan!

➡ When we are scanning machines that are not ours, we often want to hide our IP (our identity). However, we still need to ensure that packets contain our source address; otherwise, the response from the target system will not know where to return to.

➡ If we spoof our IP address in Nmap, the victim IP will respond to the spoofed IP, and not us. This is not very useful for information gathering.

➡ A better solution is to obfuscate our IP address by using the **-D** option. Nmap allows us to use decoy IP addresses so that it looks like many IP addresses are scanning the target.

```
➤ sudo nmap -sS 192.168.144.129 -D 10.0.0.1,10.0.0.2,10.0.0.4
```

➡ 💡 It is worth noting that this option does not really hide your IP, but buries it among many IP addresses so that the network/security administrator cannot easily pinpoint the source of the scan. Hackers and pentesters usually use proxies for full anonymity. One proxy that is pre-installed on Kali called **proxychains** can be used to hide your identity completely when in NAT.

5.3 Specific Ports

➡ In most cases, especially when you are conducting enumeration on certain services, you may want to scan specific ports rather than the full range.

➡ The **-p** option allows us to instruct Nmap to scan specific ports, or a specific range of ports.

↳ We can explicitly specify which port to scan on for either a specific host or the whole network.

```
➤ nmap -p 22 192.168.144.129
```

↳ This command will scan port **22** for the SSH service on a specific host, in this case the OWASP machine.

↳ We can also scan port **22** on the whole network range (particularly live hosts will respond), and identify which host is using SSH.

```
➤ nmap -p 22 192.168.144.*
```

↳ In the next command, we explicitly instruct Nmap to scan ports between two values rather than the full range, in this case ports between 1 and 1000 (the first 1000 ports).

```
➤ nmap -p 1-1000 192.168.144.129
```

↳ We now scan only two ports instead of a port range. We use the HTTP server ports: **80** and **443**.

```
➤ nmap -p 80,443 192.168.144.129
```

5.4 Reason

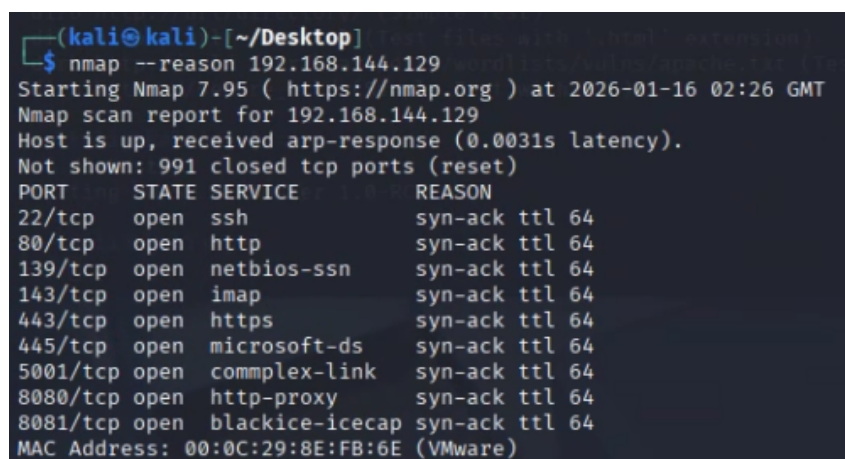
➡ The **reason** option instructs Nmap to return the reason why it has placed a particular port in a specific state.

➡ This is useful when you obtain incomplete information about a port and need more detail.

```
❏ nmap --reason 192.168.144.129
```

➡ This command provides additional information about the port state and explains why it was reported in that state.

➡ In this case (Figure 5.2), Nmap reports that the open port state is due to the response from OWASP to the SYN message with an ACK message.



```
(kali㉿kali)-[~/Desktop]
$ nmap --reason 192.168.144.129
Starting Nmap 7.95 ( https://nmap.org ) at 2026-01-16 02:26 GMT
Nmap scan report for 192.168.144.129
Host is up, received arp-response (0.0031s latency).
Not shown: 991 closed tcp ports (reset)
PORT      STATE SERVICE      REASON
22/tcp    open  ssh          syn-ack ttl 64
80/tcp    open  http         syn-ack ttl 64
139/tcp   open  netbios-ssn  syn-ack ttl 64
143/tcp   open  imap         syn-ack ttl 64
443/tcp   open  https        syn-ack ttl 64
445/tcp   open  microsoft-ds syn-ack ttl 64
5001/tcp   open  complex-link syn-ack ttl 64
8080/tcp   open  http-proxy   syn-ack ttl 64
8081/tcp   open  blackice-icap syn-ack ttl 64
MAC Address: 00:0C:29:8E:FB:6E (VMware)
```

Figure 5.2: Default Nmap scan with reason **option** return the full three way-handshake as the reason for open state port.

5.5 Merging Options

➡ We can merge options in Nmap, for example we can conduct enumeration for both services and the operating system at the same time.

```
❏ nmap -sV -O 192.168.144.129
```

↳ -sV instructs nmap to use version detection to detect the version of the service running behind a port.

↳ -O tells Nmap to try to guess the operating system running on the target using the information collected from open ports and versions

➡ Another especially useful example is using the **--reason** option when conducting UDP scans.

➡ Because UDP does not acknowledge receipt of packets, we wouldn't not know the reason for a state reported by nmap.

➡ In the output from the UDP scan above, we noted that some ports were reported as open/filtered. This meant that we couldn't determine whether the port is open or it is filtered by a device such as a firewall.

```
➤ nmap -sU --reason 192.168.144.129
```

↳ Nmap returned the same results, but it gave us the reason of the particular state of the port.

💡 You can merge any options together!

5.6 Output

➡ Sometimes, you need to save the output of the scan result to use later.

➡ Nmap allows users to save the output in different formats (**-oN** normal output, **-oX** XML file, **-oG** Grepable file, **-oJ** JSON file, and others).

```
➤ nmap -p 22 192.168.144.* -oG ~/Desktop/myresults.txt
```

↳ This will instruct Nmap to save the file in Grepable format on the Desktop in a file called myresults.txt. I find the Grepable format the most useful as it makes the output easy to filter using command-line tools.

↳ 💡 The symbol ~ means the home directory. This is the same as navigating to /home/kali. Kali is the username in this case.

6 Nmap Scripting Engine (NSE)

➡ In addition to the scanning and enumeration methods we have used before, Nmap offers through its **Nmap Scripting Engine (NSE)** more flexibility, as it allows us to run more sophisticated scans using a large collection of scripts written for specific purposes.

➡ On Kali Linux, Nmap Scripting Engine (NSE) scripts are installed in the default directory where Nmap is installed, which is **/usr/share/nmap/scripts/**.

```
➤ ls /usr/share/nmap/scripts
```

➡ Listing the script files in the folder reveals a large number of pre-installed scripts (607 at the time of writing!).

➡ From the script names you can see that the scripts are not only for scanning, discovery, enumeration, and vulnerability detection, but some can also be used to conduct DoS and brute-force attacks or even exploit certain vulnerabilities.

➡ The Nmap NSE also allows users to write their own scripts and share these scripts with other users for the purposes of networking, reconnaissance, and other activities.

6.1 Apache Server Enumeration Using NSE

➡ Although we have previously identified versions of services running, NSE scripts in most cases provide more detailed information about those versions.

➡ We have already identified that Apache HTTPd 2.2.14 is running on port 80 and port 443. Let us try to get some more information about this.

➡ We will use a script called **banner**. This script captures information about the service such as the version, but with more detail.

```
➤ nmap -sV --script banner -p 80,443 192.168.144.129
```

↳ This command instructs Nmap to conduct version detection and provide a detailed banner about the services running.

↳ You can see that Nmap provides a lot more information about the Apache server, including the different modules that are enabled within the Apache service. Each module can potentially be vulnerable and provide a point of entry.

➡ There are several other NSE scripts related to the Apache server that you can explore. In particular, try these scripts:

➡ **http-title**, **http-headers**, and **http-sitemap-generator**

↳ For example:

```
➤ nmap --script http-headers -p 80,443 192.168.144.129
```

➡ Check the scripts folder:

```
➤ ls /usr/share/nmap/scripts/http*
```

↳ This will show all the scripts relevant to the Apache HTTP service.

6.2 Other services enumeration

➡ We have found many services running on the OWASP machine through our initial scans. We can use Nmap NSE scripts to enumerate those services in more detail.

➡ Let's first look for NSE scripts to enumerate the SSH service on port 22.

```
➤ ls /usr/share/nmap/scripts/ssh*
```

↳ This will show all the scripts relevant to the SSH service.

↳ Note that **ssh-brute** will conduct brute-force attacks on the OWASP machine. Running this at this stage risks detection.

```
➤ nmap --script ssh2-enum-algos -p 22 192.168.144.129
```

↳ This will enumerate the SSH certificates, including the algorithms used. This can be useful as the server may be using old or vulnerable algorithms.

```
➤ nmap --script ssh-auth-methods -p 22 192.168.144.129
```

↳ This script enumerates the authentication methods supported by the server.

➡ Another service running on the OWASP machine is the IMAP service on port 143.

↳ We can use NSE scripts to enumerate the IMAP service.

```
➤ nmap --script imap-capabilities -p 143 192.168.144.129
```

```
➤ nmap --script imap-auth-methods -p 143 192.168.144.129
```

➡ The OWASP machine is also running a Tomcat web server on port 8080.

```
➤ nmap --script http-title -p 8080 192.168.144.129
```

```
➤ nmap --script http-enum -p 8080 192.168.144.129
```

```
➤ nmap --script http-auth -p 8080 192.168.144.129
```

Enumeration activity for hackers and pentesters

➡ A hacker or a pentester would typically spend significant time enumerating all services found on a target system.

💡 You should also look for relevant NSE scripts for any other services you discovered running on the OWASP machine.

7 Other scanning and enumeration tools

➡ While we focused on Nmap in this lab as it is the most popular tool and is the most widely used, it is not the only port scanner available.

➡ There are some other alternatives included in Kali Linux. We present some examples below. There are many others you can explore.

➡ One in particular that is interesting in how it represents the ARP table and discovers devices is Netdiscover, but there are others pre-installed on Kali.

➡ Netdiscover uses the ARP table to discover devices by sniffing ARP requests and responses.

```
➤ sudo netdiscover -i eth0 -r 192.168.144.0/24
```

↳ The -i eth0 means that we are sniffing on a specific network interface (eth0).

➡ There are several scanners and mapper tools that have a graphical interface. Zenmap is a popular GUI alternative to Nmap.

```
➤ zenmap
```

↳ Zenmap executes Nmap commands in the background and presents the results in a

graphical interface.

7.1 Samba enumeration

Why Samba enumeration?

➡ Our initial scans revealed that port 445 is open and the Samba service is running (Fig.4.2). Samba is a service that allows users to share folders, printers, and other resources on both Windows and Linux systems. If Samba is not configured properly, enumeration can reveal a large amount of information.

- ➡ Let's see what information we can obtain by enumerating the Samba service.
- ➡ The **enum4linux** tool is designed specifically for Samba service enumeration.
- ➡ Let's first look at the command options in the help page of the tool, including the various options we can use.

```
>- enum4linux -h
```

- ➡ We now attempt to enumerate user accounts on the OWASP machine.

```
>- enum4linux -U 192.168.144.129
```

- ➡ This will enumerate users on the OWASP machine. This command will find three users, including the one we already know about (See Figure.7.1).

```
( Users on 192.168.144.129 )
index: 0x1 RID: 0x1f5 acb: 0x00000010 Account: nobody Name: nobody Desc:
index: 0x2 RID: 0x3e8 acb: 0x00000010 Account: user Name: user,, Desc:
index: 0x3 RID: 0x3e9 acb: 0x00000010 Account: root Name: root Desc:
user:[nobody] rid:[0x1f5]
user:[user] rid:[0x3e8]
user:[root] rid:[0x3e9]
enum4linux complete on Fri Jan 16 02:53:48 2026
```

Figure 7.1: Samba service users enumeration

- ➡ Next, we retrieve general machine information from the Samba service.

```
>- enum4linux -M 192.168.144.129
```

- ➡ We now attempt to list shared folders using default credentials.

```
>- enum4linux -S 192.168.144.129
```

- ➡ This will attempt to map shared folders using default credentials but will result in access being denied.

- ➡ We now retrieve the password policy enforced by the Samba service.

```
>- enum4linux -P 192.168.144.129
```

- ➡ This will reveal the password policy, including length and complexity requirements.

➡ Next, we enumerate group memberships and domain information.

```
➤ enum4linux -G 192.168.144.129
```

↳ This will reveal group membership and the domains users belong to.

➡ We now perform a full enumeration of the Samba service.

```
➤ enum4linux -a 192.168.144.129
```

↳ This will reveal information about user accounts, shares, groups, password policy, and more.

➡ Finally, we use valid credentials to enumerate accessible shared folders.

```
➤ enum4linux -u root -p owaspbwa 192.168.144.129
```

↳ This maps shared folders using the username and password we already know. It reveals additional services that were not previously identified, such as the Messenger service.

A note on Samba enumeration

➡ We could have used the Nmap SMB scripts, but the **enum4linux** tool is more focused and its output is easier to read.

```
➤ ls /usr/share/nmap/scripts/smb*
```

💡 This is the end of the lab, and we only covered a few of the usual activities that take place during the scanning and enumeration phase. As you can see, the more in-depth the analysis is, the more valuable information a hacker or penetration tester can uncover, which can be used to exploit target machines.